Wildlife Playback Speaker

University of California – Santa Cruz
ECE 129 Capstone Series
Senior Capstone Report
Bachelor of Science in Robotics Engineering

Prepared by: Caitlin Bonesio, Ramsey Mogannam, Jaspreet Singh

Sponsored by: Mike Kowalski
May 2025

Abstract

This report summarizes the design, development, and testing of a wildlife playback speaker system aimed at enabling long-duration outdoor deployment for animal behavior research. The system integrates audio playback, scheduling, and energy-efficient operation.

Table of Contents

1. Introduction

1.1 Project Background

The Wildlife Playback Speaker project is part of the UCSC ECE 129 Capstone Series. The goal is to support ecological field studies by creating a robust, autonomous playback device.

1.2 Objectives

Our primary objectives were to achieve reliable audio playback, low power consumption, and a user-friendly interface for scheduling and control.

2. Project Purpose and Goals

2.1 User Needs/Technical Requirements

Researchers require a device capable of playing pre-recorded sounds at scheduled intervals in remote environments with minimal maintenance. The system must support Bluetooth and UART connectivity, have weatherproof housing, run on battery power for over 24 hours, and support WAV file playback.

3. System Overview

3.1 Hardware Architecture

The hardware includes a microcontroller, SD card module, audio amplifier, speaker, buck converter, and rechargeable battery system.

4. Key Improvements Over Previous Design

4.1 Embedded Subsystem

Lead: Caitlin Bonesio

In the previous design, the system utilized an ESP32 microcontroller due to its small size and ability to communicate with the MP3 player. However, this configuration posed limitations in power efficiency, no access to internal flash memory, and stability for Bluetooth Low energy (BLE) communication in the field. To address this, we replaced the ESP32 with the STM32WB05KZ, a microcontroller optimized for BLE applications.

The previous team's software design was never made available, necessitating the current software be built from the ground up. The STM32WB05KZ manages several communication methods, including UART, USART, I2C, Bluetooth, and GPIO communication. The microcontroller (MCU) also handles system logic such as duty cycle control, scheduling, logging, and managing peripherals. All of these features were designed from a software block diagram.

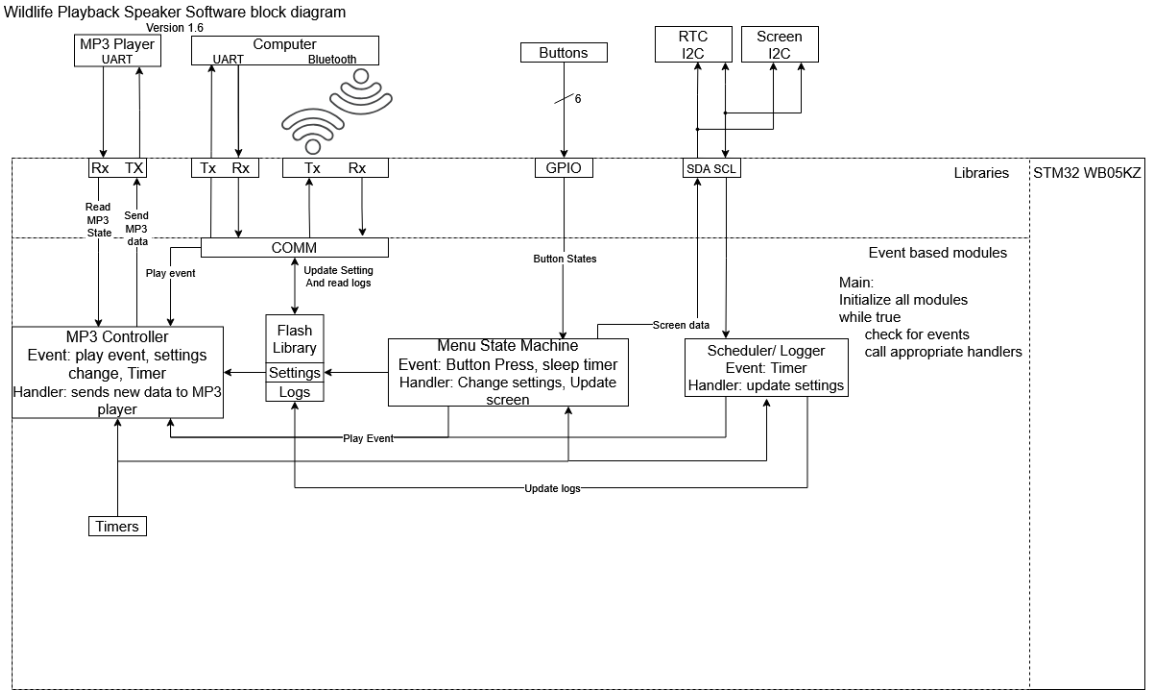


Figure 4.1.1: Software block diagram

This modular nature is supported by an events and services routine. This allows each module to allow others to run while waiting for an input. This allows for an abstraction that each module operates independently and asynchronously. Each module communicates with each other and themselves through events. Each module has a FIFO queue of events that the updater or other modules can post to. These events are then processed by the module handler.

4.1.1 UART

The UART library is used for controlling the internal low power UART and USART module on the STM32. The library has 5 public functions. The first is an initialization to configure the internal peripherals. The read and write functions each interact with their own RX and TX circular buffers, one of each type for the UART and the USART. Private interrupt functions write to the appropriate RX buffer when new data is detected and reads from the appropriate TX buffer when the associated transmit line is available.

The LPUART module is used to send and receive packets to and from the MP3 player to control what it plays. The MP3 player is initialized by sending the restart command (0x0C) then the STM queries the number of tracks in each folder of the MP3 player with the 0x4E command. The MP3 player will either respond with the same query command to indicate the packet data has the number of folders or it returns an error (0x40). Once the STM receives an error it knows that all folders have been scanned. The internal folder map is then constructed and stored on the heap for later. When the STM wishes to play a folder and track, it sends the play track packet (0x03) with the track number being represented by the number of all tracks in all folders previous to the desired track and folder in the folder

map. This is done to side step the unreliable play track/folder command (0x0F). Similarly, the repeat folder command is unreliable; to sidestep this the MCU receives track complete from the MP3 to count until it reaches the end of the folder in its internal folder map and sends a play track command for the first track in the folder. This implements the track/folder play and folder repeating requested by the client.

4.1.2 USART/Bluetooth

The USART and Bluetooth both handle communication with an external laptop or desktop graphical user interface (GUI) application. The COMM module polls the USART RX buffer for incoming data. The Bluetooth is managed by an STM32 middleware. The Bluetooth is configured to transfer sequences of single bytes to and from the GUI in order to use the same parsing state machine as the USART. It does this with three characteristics: VIRTUAL_USART_TX, VIRTUAL_USART_RX, and VIRTUAL USART_TX_REQ. The TX, and RX are named from the microcontroller's perspective, the data that flows from the MCU to the PC is in the TX characteristic, and vice versa for the RX. TX_REQ is used to make the act of the MCU transfering to the PC atomic. This means the PC must confirm the received data in TX_REQ for the MCU to post more. For the RX characteristic the middleware is directed to post to the COMM module state machine the moment the RX is changed.

The COMM module parses the incoming data packets from the USART and Bluetooth. The packet format the team agreed on was sending a command byte followed by a predetermined number of bytes required to fulfill that command. Some commands require a variable number of bytes to complete, these use a done command to indicate the end of transmission. The predetermined commands are: volume control, folder control, logs request, logs done, duty cycle control, schedule control, schedule end, debug print, debug end, and set time. These implement the ability for the PC to set the volume, duty cycle, schedule and time and implements the MCU's ability to send the logs and debug text.

4.1.3 I2C

The I2C library is used to control the internal I2C peripheral. The library is built to interact with the event based architecture of the system. This means every receive request has an associated post function to post the received data to. This allows the receive functionality to be non-blocking and non-polling. The core of the I2C module is a buffer storing both receive and transmit requests. The buffer can be loaded with receive and transmit requests that will be pulled from by the I2C interrupt. The module uses the HAL i2c memory interrupt functions and callbacks to implement this. The buffer enqueue function has to be able to differentiate between the receive and transmit requests to call the appropriate equations.

The I2C peripheral is used to control the real time clock (RTC) and OLED screen. The RTC is initialized by reaffirming the start bit is enabled and the use back battery bit is enabled. then receive requests are used to read from the registers. The data in the registers are in binary coded decimal (BCD) format. The MCU must convert these BCD numbers into pure binary format. The OLED screen requires far more initialization steps. The display is turned off, then the multiplex ratio, display offset, start line, segment remap, com output

scan direction, and com hardware config is set. Then the display is turned on. To control the screen, a UCSC OLED controller library was used. The OLED library required new initialization code, a wrapper function in the I2C library, and the output had to be remapped to flip the display upside down for better ease of use.

4.1.4 GPIO

The GPIO library provides initialization to read from the PCBs buttons. The library provides functions to initialize the pins and to poll from the pins. Both functions are wrapper functions for the HAL GPIO functions

The GPIO pins are used for the buttons menu state machine (BMSM). The BMSM uses a state machine to implement a menu for the user to navigate using the buttons. It provides the ability to play certain tracks and folders, schedule upcoming folder changes, set the devices time, seek forward and back, change the volume, duty cycle and clear the schedule. With exception to the main menu, button one is always mapped to back, button two is always mapped to select, button three to move the cursor down, button four to up, button 5 to left, and button 6 to right. By keeping the buttons consistent, it is easier for a user to learn the system.

4.1.5 Duty Cycle control

The MP3 module is in control of maintaining the speakers playing duty cycle. This is done by using the free running timer (FRT) from the TIMERS library. The module has the full cycle length predefined. In the initial play state the full cycle length is multiplied by the duty cycle as a percentage and compared against the FRT. When the timer has reached the end of the duty cycle it swaps into the paused state where the FRT is compared to the full cycle length multiplied by one minus the duty cycle. At the end of the pause state it returns to the play state. The module is built to be able to continue from where it left off from pausing.

4.1.6 Scheduling

The scheduling functionality is implemented by having the module produce an event every minute to check the RTC. This time is compared to the schedule that is stored on the flash memory. If the current time is within the scheduled time and the MP3 isn't already playing the same track and folder, the Scheduler module posts a play event to the MP3 controller to change the track and folder. The scheduler is set up like this to be able to recover from a shut down as the start time is missed but the MP3 player should be playing.

4.1.7 Logging

The logger module is designed so that the MP3 player forwards all received play events to the logger. When the logger receives a play event, it requests the time from the RTC and appends the log with the track, folder, and time.

4.1.8 I2C-USART manager

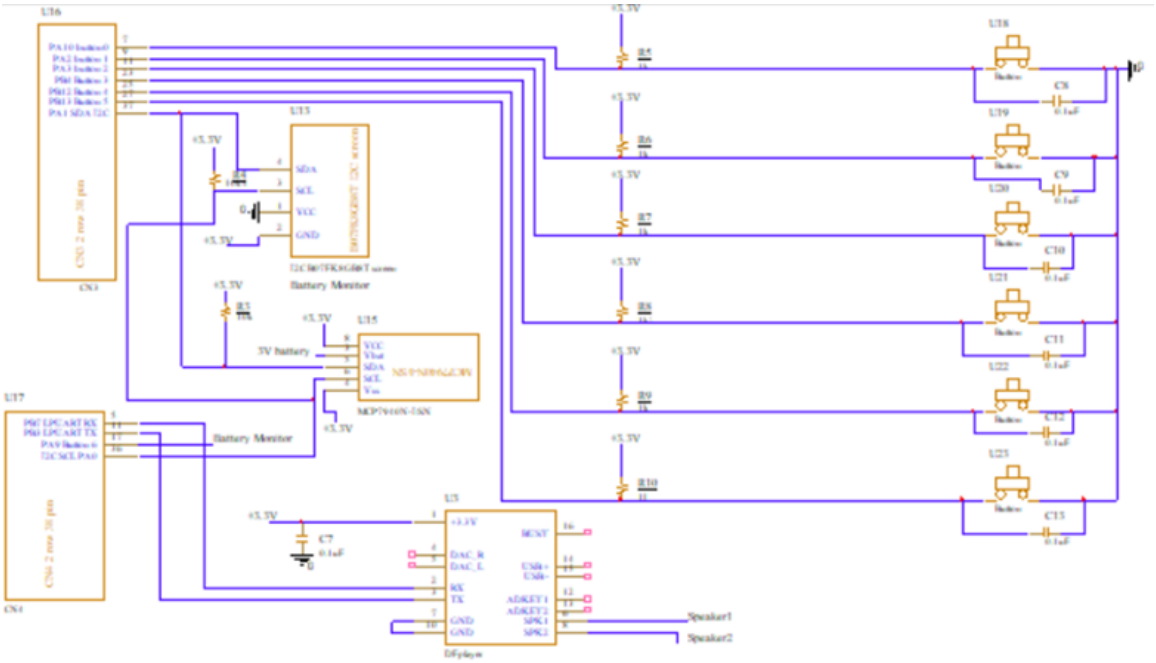
The system had a design flaw where both the I2C peripheral's serial data line (SDA) and USART peripheral's TX line are both mapped to the pin PA1. This conflict required a

separate manager library to provide arbitration between the two peripherals. The default state is letting the I2C SDA line control PA1. When the USART library is to transmit, the manager swaps the pin from I2C SDA to USART TX and when the TX buffer empties, the control is returned to SDA. The manager also disables the unused peripheral while it does not have control to prevent data being lost to an unconnected pin. This flaw needs to be fixed in any future interaction

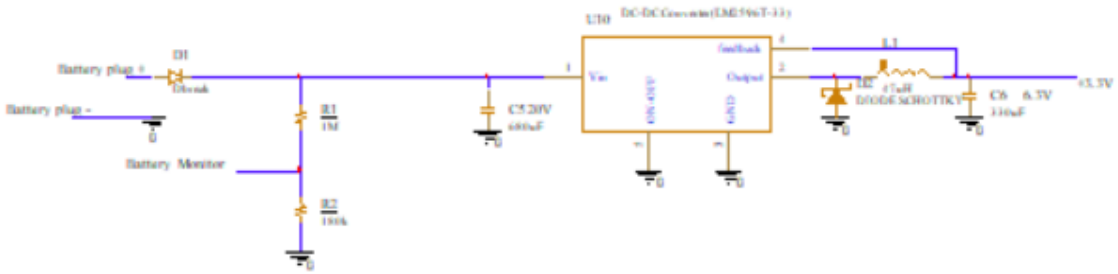
4.2 PCB Subsystem

Lead: Ramsey Mogannam

To start the PCB section of this project, I first started with creating the schematics. I first started by creating the schematics for the project. I first started by downloading Capture OrCAD, as that is the program I first used to create the schematics. This is my first time doing a schematics or creating any circuit board. Hence, it has been a huge learning curve and the first trial of the schematics is shown below:-

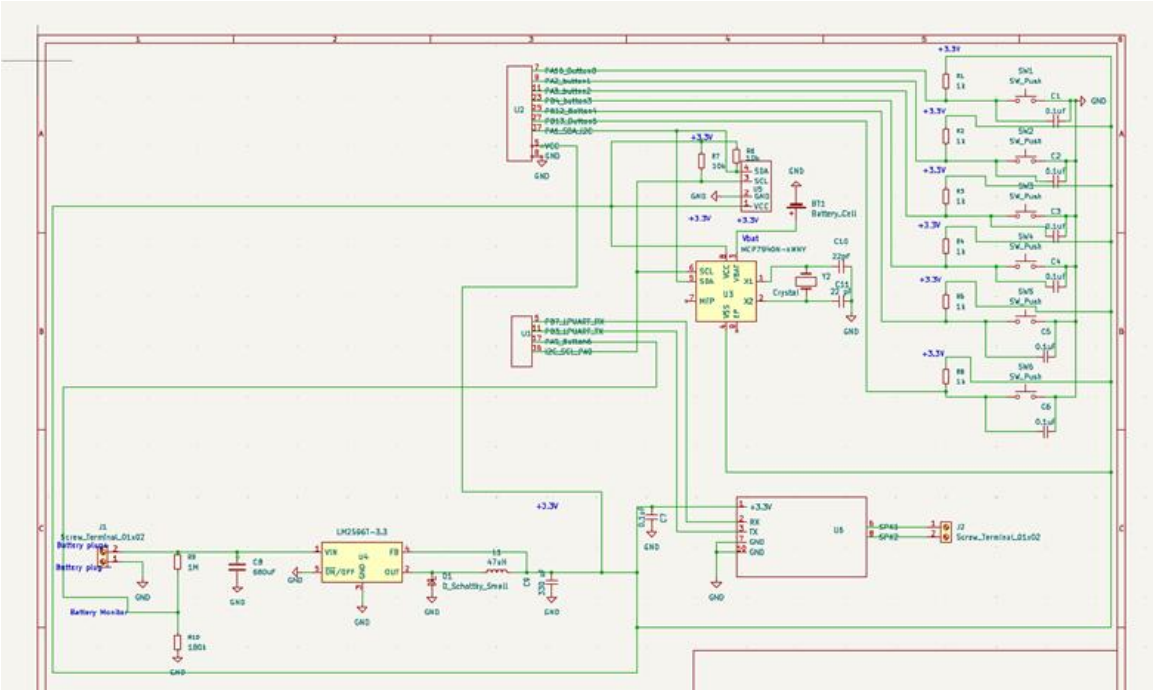


The buck converter is identical to the one from the first gen design. It is shown in the figure below:-

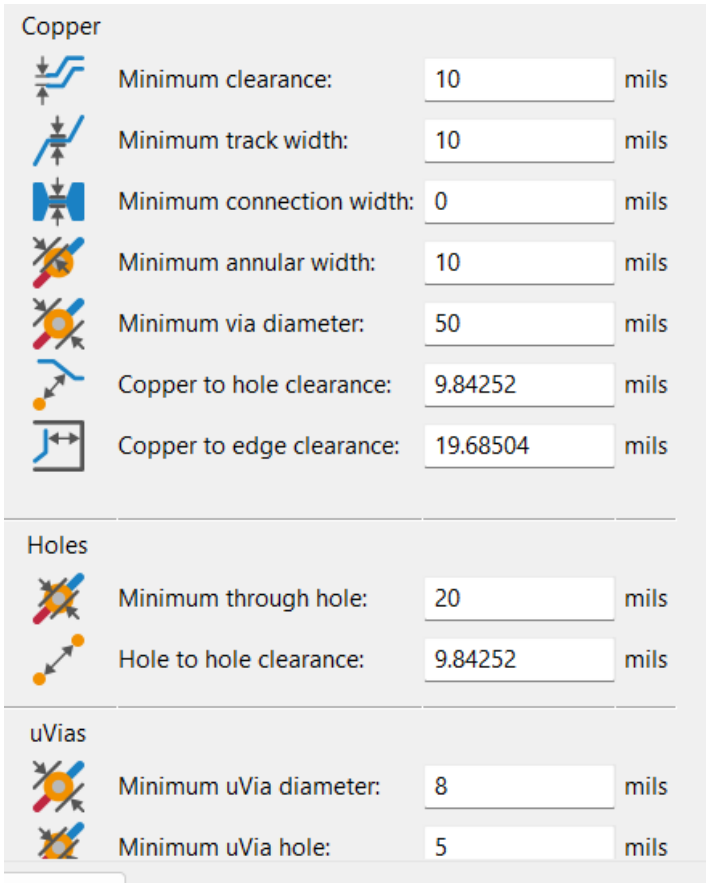


Since the DC-DC converter was working perfectly, there was no reason to alter or modify it.

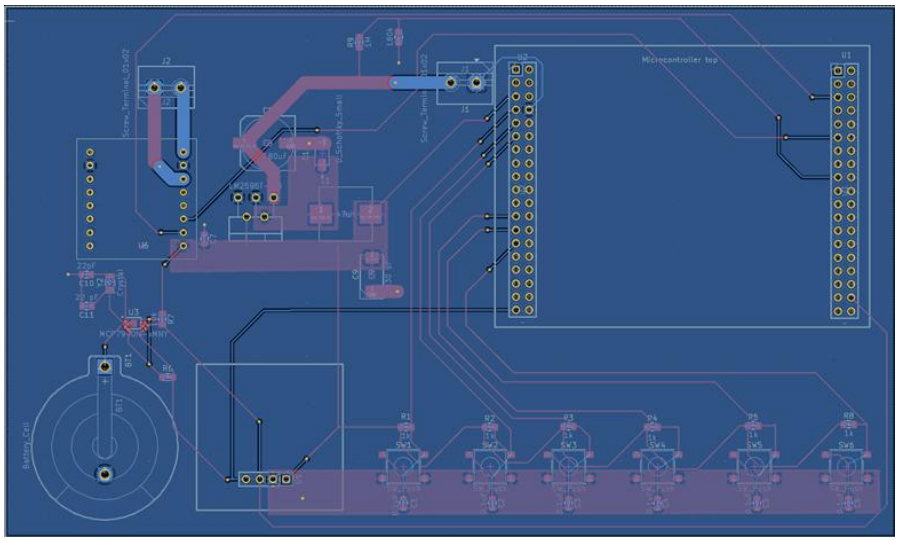
After the first trial of the schematics was completed, I tried adding the PCB footprints which is what will be used to create the layout. Here I ran into a lot of technical issues as windows 11 is not compatible with Allegro. Hence, I had to switch to using Kicadd, as a result I had to recreate the schematics using KiCadd. This was beneficial as Kicadd has a library downloaded with all of the components we will be using. To add a footprint, left click on the component and select the assign footprint function. The schematics using Kicadd is shown below:-



Some of the pins have been changed, also the power and ground pins for the microcontroller were added. We also proceeded to create the PCB layout for this board. To start the PCB layout, it is very important to setup the constraints on KiCadd. The constraints are shown below:-

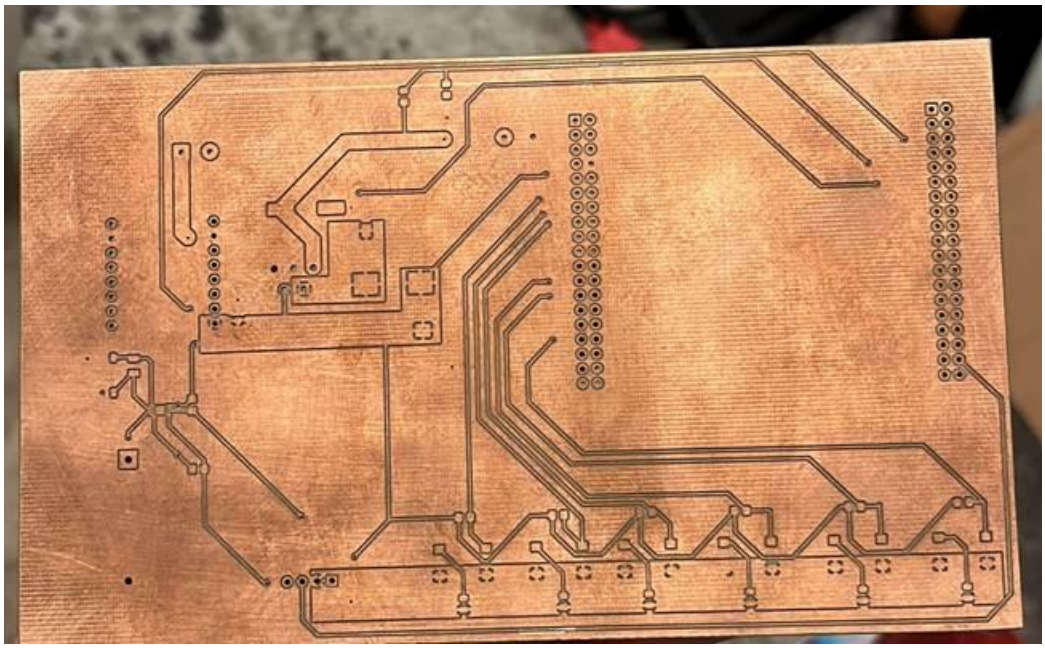


The constraints are shown above, the minimum clearance between each track is 10 mils, the track width is set for 10 mils.



The first version of the board is shown above. All traces other than high power ones have a thickness of 10 mills. The vias have an annular width (diameter) of 50 mils and the via hole is 20 mils. A micrometer was used to measure the distance between the microcontroller pins. This was done to make sure the microcontroller can sit accurately on the pin headers.

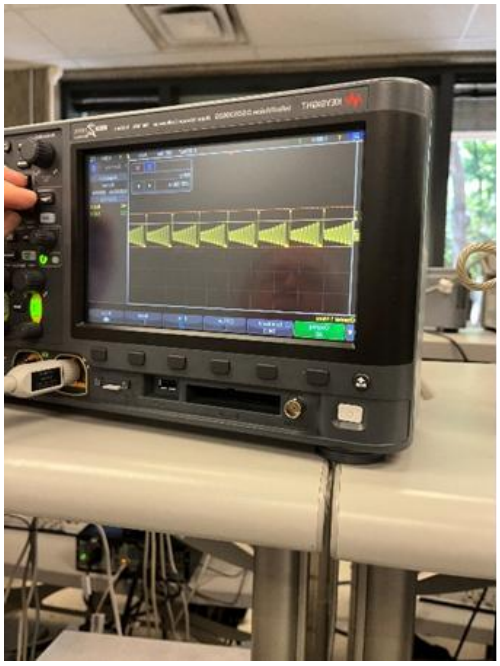
A copper area was added to the back of the board to create a ground plane. For the high power area, (the area between the voltage input and the LM2596), copper area and a thick trace of a 100 mils was used. This was done as these areas carry a lot of current and that would prevent the traces from getting damaged. We proceeded to mill the board. After the milling is completed the board is shown in the picture below:-



I proceeded with assembling the board, and it is shown in the picture below:-

Testing and Debugging of the first iteration of the PCB:-

It is very important to test the board while assembling it. This was a mistake I did, as I assembled the whole board then proceeded with testing it as one unit. To test the board while assembling check for continuity between pins to make sure the components are not shorting. I ran into an issue where the DC-DC converter was not producing 3.3V, after a few days of testing. It ended up being the diode was installed backwards. To make sure the DC-DC converter is working, measure the voltage on the copper area and 3.3V should be seen. The current draw when the board if working perfectly according to the LM2596T-3.3 data sheet is from 10-40 mA. When the board is connected and powered it draws 15mA, hence, the board is functioning properly. Also, connected an oscilloscope to the output and the result is shown in the pictures below:-



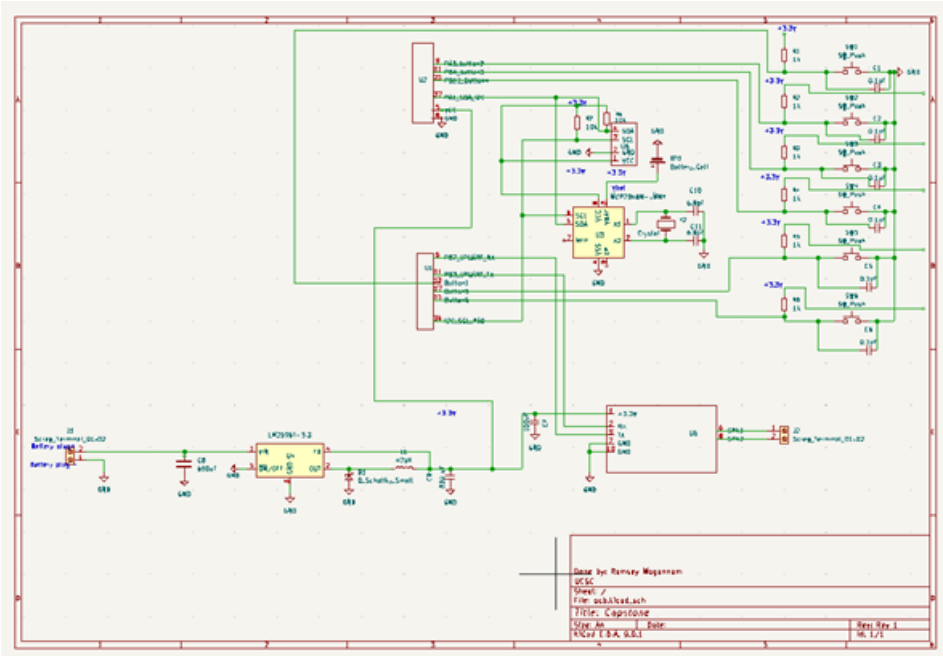
To test the buttons and make sure they are working, connect the oscilloscope probe the output pins on the pin headers, when the button is unpressed, the output of the oscilloscope should be seen as below:-

The output of the oscilloscope when the button is pressed is shown in the picture below:-

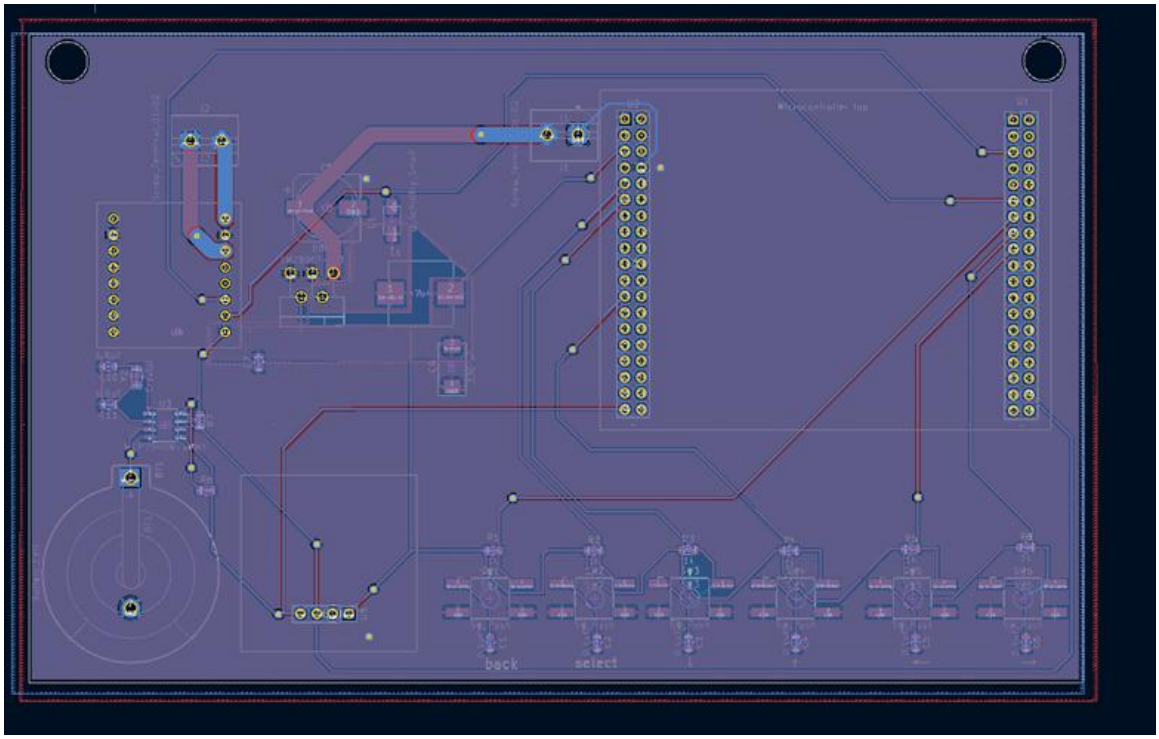
as seen in the picture above, there is a change in the output of the oscilloscope which shows that the button is working properly.

Technical difficulties:-

We ran into some technical difficulties as the pins on the microcontroller were not correct. The STM datasheet was not correct, hence, I had to readjust the schematics and layout to accommodate for the pins readjusting. The schematics is shown in the picture below:-



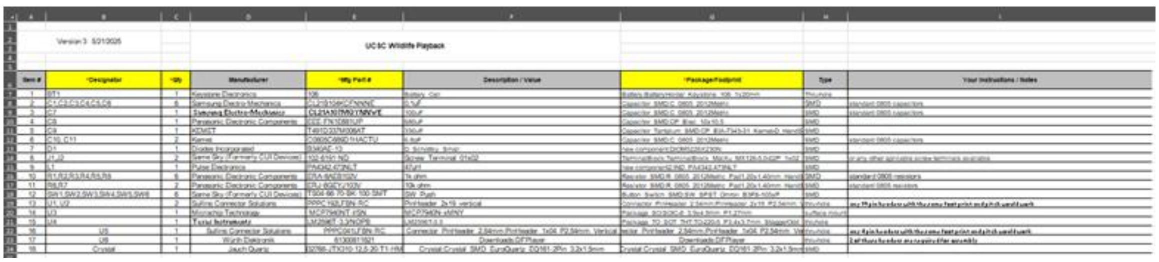
The PCB layout is shown in the diagram below:-



I also proceeded with adding a copper area for the ground plane on the front. Capacitor values for the Real Time Clock(RTC) were changed to 6.8pF(C10,11). This was done as the RTC was skipping time and losing about 3 minutes every hour.

PCBWay:-

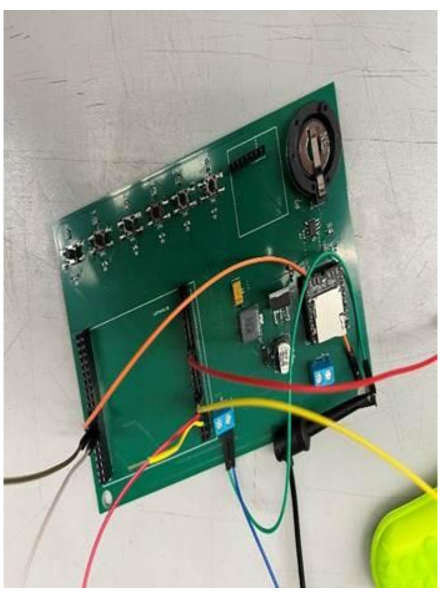
As per clients request, we proceeded to get the board manufactured by PCBWay, it costed around $7 per board. To manufacture and assemble a board it costs $40 with components. I created a Bill Of Materials (BOM) and choose all the components desired to assemble the board using Digikey parts. A picture of the BOM is shown below:-



Version 3 5/21/2025 — UCSC Wildlife Playback

Item #	*Designator	*Qty	Manufacturer	*Mfg Part #	Description / Value	*Package/Footprint	Type	Your Instructions / Notes
1	BT1	1	Keystone Electronics	106	Battery Cell	Battery BatteryHolder Keystone 106 1x20mm	thruhole	
2	C1,C2,C3,C4,C5,C6	6	Samsung Electro-Mechanics	CL21B104KCFNNNE	0.1uF	Capacitor SMD:C 0805 2012Metric	SMD	standard 0805 capacitors
3	C7	1	Samsung Electro-Mechanics	CL21A107MQYNNWE	100uF	Capacitor SMD:C 0805 2012Metric	SMD	standard 0805 capacitors
4	C8	1	Panasonic Electronic Components	EEE-FN1C681UP	680uF	Capacitor SMD:CP Elec 10x10.5	SMD	
5	C9	1	KEMET	T491D337M006AT	330uF	Capacitor Tantalum SMD:CP EIA-7343-31 Kemet-D Hand	SMD	
6	C10, C11	2	Kemet	C0805C689D1HACTU	6.8pF	Capacitor SMD:C 0805 2012Metric	SMD	standard 0805 capacitors
7	D1	1	Diodes Incorporated	B340AE-13	D Schottky Small	New component:DIOM5226X230N	SMD	
8	J1,J2	2	Same Sky (Formerly CUI Devices)	102-6161-ND	Screw Terminal 01x02	TerminalBlock TerminalBlock MX126-5.0-02P 1x02	SMD	or any other applicable screw terminals available
9	L1	1	Pulse Electronics	PA4342.473NLT	47uH	New component:IND PA4342.473NLT	SMD	
10	R1,R2,R3,R4,R5,R8	6	Panasonic Electronic Components	ERA-8AEB102V	1k ohm	Resistor SMD:R 0805 2012Metric Pad1.20x1.40mm Hand	SMD	standard 0805 resistors
11	R6,R7	2	Panasonic Electronic Components	ERJ-8GEYJ103V	10k ohm	Resistor SMD:R 0805 2012Metric Pad1.20x1.40mm Hand	SMD	standard 0805 resistors
12	SW1,SW2,SW3,SW4,SW5,SW6	6	Same Sky (Formerly CUI Devices)	TS04-66-70-BK-100-SMT	SW Push	Button Switch SMD:SW SPST Omron B3FS-105xP	SMD	
13	U1, U2	2	Sullins Connector Solutions	PPPC192LFBN-RC	Pinheader 2x19 vertical	Connector Pinheader 2.54mm:Pinheader 2x19 P2.54mm	thruhole	any 19 pin headers with the same footprint and pitch would work
14	U3	1	Microchip Technology	MCP7940NT-I/SN	MCP7940N-xMNY	Package SO:SOIC-8 3.9x4.9mm P1.27mm	surface mount	
15	U4	1	Texas Instruments	LM2596T-3.3/NOPB	LM2596T-3.3	Package TO SOT THT:TO-220-5 P3.4x3.7mm StaggerOdd	thruhole	
16	U5	1	Sullins Connector Solutions	PPPC041LFBN-RC	Connector Pinheader 2.54mm:Pinheader 1x04 P2.54mm Vertical	Connector Pinheader 2.54mm:Pinheader 1x04 P2.54mm Vertical	thruhole	any 4 pin headers with the same footprint and pitch would work
17	U6	1	Würth Elektronik	61300811821	Downloads:DFPlayer	Downloads:DFPlayer	thruhole	2 of these headers are required for assembly
18	Crystal	1	Jauch Quartz	Q32768-JTX310-12.5-20-T1-HMR	Crystal:Crystal SMD EuroQuartz EQ161-2Pin 3.2x1.5mm	Crystal:Crystal SMD EuroQuartz EQ161-2Pin 3.2x1.5mm	SMD	

Since manufacturing and assembling the board requires a lead time of 30 days, I proceeded to order the board without assembly. After a few days, I received the board unassembled and proceeded with assembly. A picture of the board after assembly is shown below:



Testing:-

I tested the board the exact same way I tested the first iteration of this PCB, and this board works perfectly.

PCBWay DFM:-

I proceeded with uploading the final PCB after testing it with the BOM to PCBWay so that the client can proceed with ordering all 40 boards that he needs to complete his project.

Lessons learned and challenges faced:

Creating the PCB has been a huge learning experience since I have never done a PCB design before in my life. I faced a lot of challenges with understanding the design standards for PCBs. I learned how to design a PCB. In addition, I faced a lot of challenges with debugging the PCB. As a result, this taught me a lot on how to debug circuits and how to utilize oscilloscopes. I faced a lot of challenges with relaying on the datasheets since it was proven that some of the datasheets were inaccurate. This taught me to not trust a datasheet and to test and verify that a datasheet is accurate before designing the PCB.

Future work:-

I will be editing the PCB layout to make it smaller, so that it can fit better on the battery mount.

4.3 Mechanical Subsystem

Lead: Ramsey Mogannam

The client reported that there were numerous issues with the first generation design of the box. The box was not waterproof, there was water leaking from the wire glands and the speaker present on the box. Client has also mentioned that the battery was not secure and it caused damage to the PCB as it was moving around. Client also mentioned there is a lot of resonance present in the box in tracks with high frequency noises. After noting all of these issues, I proceeded with testing the waterproofing on one of the old box to confirm that the wire glands and speaker were leaking.

Testing of old box:-

After getting an old box from the client, I proceeded with using a hose to simulate rain and it was

confirmed that the wire glands and speaker were the source of the leak. I then proceeded to take

the old box apart to understand why the waterproofing failed. I found a few reasons why the

glands were leaking. First of all, the glands were missing o-rings, and the wrong size glands were

used. PG11 glands were used which are too big for a 5mm wire. Also, the holes drilled in the box

were not drilled correctly, it seemed like the drill was not held straight when the holes were

drilled which caused the box to form a bad seal.

Fixing the waterproofing on the box:-

To fix the waterproofing on the box, I proceeded with ordering a new box and building a new case the right way.

Step 1:-

Order the box, the box used during this project is the 230 Meduim Tactical Case, which can be found on amazon using the link below:-

https://www.amazon.com/Seahorse-Medium-Tactical-Case-Black/dp/B06ZXXX29F/ref=sr_1_3?crid=XRX90MKTKH9U&dib=eyJ2IjoiMSJ9.CXgwq6q43-RZNpyl7hS4Y7P_fFGsPEHgXrmiyL0R7wpFvRBqD8a80hxzv13scyqVZ1taLV7v0uIJRUCGLiVxUIxrtVZzi-vkNTomRZYBtTFdEObxXpFrNT3vrWZB7iLgJNdYIrK1EX-ujiBhPSNZ1-qaQEpjHKlV-hYU5cegBaS0Y4BHiUZXjGUIxt57EKIVjS1r0PVyEepXQOL2jXZ70yC6nsEdgmCcRvtmb-jhgEM.J3bkg6NE6dIjuSlAMyYC_80nRKuuFwXQ5gWYShiQIjU&dib_tag=se&keywords=Seahorse+230&qid=1740781420&sprefix=seahorse+230%2Caps%2C182&sr=8-3

Step 2:

To start with manufacturing the box, locate the wiring glands. Since the wires we are using have a thickness of 7 mm. Since the thickness of the wire is on the lower threshold of PG7 which has a tolerance of 3mm-7mm. To drill the holes for the glands, measure 2” from the corner of the box. Then, use a drill with a bit size of 5/16. I made sure to use a drill bit that is a bit smaller than the size of the wire gland. It is very important to keep the drill straight to make sure that the glands won’t leak. A picture of the drilled holes is shown below:-

Step 3:

For the third step, we will be using a tap set to tap the case. This is done to make sure to eliminate any water leaking into the box. The tap size I used is 7/16, before installing the wire glands, make sure to thread sealer on the threads of the wire gland. Adding a small dot on the threads of the wire glands is more than enough. Refer to the diagram below for a detailed picture on how it should look. After the sealer has been applied, thread the wire gland in, and make sure to add both O-rings on both sides of the wire glands and install the nut. Install the wires, with making sure to add the rubber gromets and stoppers to make sure the wire doesn't back out on its own. Use two adjustable wrenches to tighten the gland, be careful of over tightening the nuts and the wire glands as it can break the case or the gland. Wait a few hours to let the thread sealer dry before moving to step 4. Pictures of this step are shown below:-

Step 4:

Make sure to use black RTV(automotive gasket maker that is heat resistant) and will withstand the weather conditions of the testing grounds. Use wire cutters to cut the end of the RTV tube at a 45 degree angle. Make sure to cut it very close to the edge of the closed RTV tube so that the bead won't be too thick. I spread a layer of the RTV around all the wire gland, and then made sure to spread it with my finger to confirm a tight seal around the glands. A picture of the final product of the glands is shown below. Leave the box for a few hours under direct sunlight to let the RTV cure.

Step 5:

In this step, we will be using the automotive sound insulation. This is important because the first generation of the box had a lot of resonance in some soundtracks. To fix this issue, automotive insulation will be applied to all of the insides of the box. I first started with the bottom section of the case. It is very important to clean the box using isopropyl alcohol and a lint free rag to make sure the surfaces are oil free. Wearing gloves is necessary when cleaning and handling the insulation to avoid any oils from our skin transferring onto the box. To install the insulation, peal the adhesive off the back of the insulation and use a roller to install it. Make sure to press very firmly on the roller to make sure the insulation is secure. Since the insulation is very flexible, it can be installed on the sections of the box and an exacto blade was used to cut all the access material. I repeated this for all of the sides of the box the dimensions are shown in the pictures below:

Step 6:

In this step we will be cutting the hole for the speaker and mounting it. To cut the hole for the speaker, use a hole saw bit with a size of 4”. It is going to be a bit difficult to cut through the insulation, if it gets stuck make sure to reverse the drill direction and then reverse back. Having a high RPM drill here is a life savor! To mount the speaker, the provided gasket can be used. I used RTV as I felt like it was more secure. Either way works, but make sure to add a bead of RTV on the bottom of the speaker as shown in the picture below to make a good seal. Add screws in the speaker to mount it. Make sure to hold the speaker down to avoid any movements in the speaker itself. I waited a few hours to let the RTV dry then I added another layer of RTV all around the speaker. Pictures are shown below:

Step 7:

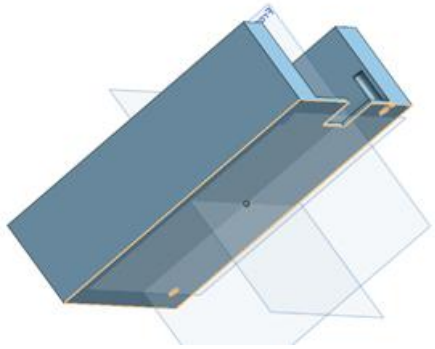
In this step, we will be working on testing the waterproofing of the box. Since the boxes aren't the best quality. Some of the boxes are not fully waterproof. To prevent any damage to components from happening, we will be testing the waterproofing using a hose. It is very important to make sure the silicon and thread sealer is completely dry before any testing.

Step 8:

I started by adding Velcro on the bottom pieced of the box to mount the battery and the AC-DC converter from the solar panel. Make sure to mount it in the center far from the edges as the battery mount will be taking up that space. Then, I proceeded to use zipties to hold all the wires in place.

Battery Mount:-

I finally proceeded with starting to create the mount for the battery using Onshape. The mount is a very simple design, and it is shown in the picture below:



The mount has two holes integrated into it, which the user can decide to install an M3 thread insert or use a self-taper screw to install the PCB on the mount.

Challenges faced and lessons learned:-

I faced a challenge of finding the best way to make the box waterproof, this taught me a lot about wire glands and waterproofing. I also learned how to use On shape, as I have never used it before, I always used SolidWorks.

5. Testing and Validation

5.1 Unit Testing

All subsystems—audio, scheduling, power, and GUI—were tested individually for expected behavior and performance.

5.2 System Testing

Integrated system testing was performed to verify correct execution of scheduled playback and long-term power reliability.

6. Lessons Learned

6.1 Challenges Faced

Initial power regulation caused unexpected resets. Debugging UART communication with Bluetooth modules also required extensive testing.

6.1.1 Embedded systems

The most consistent issue the embedded systems team encountered was poor documentation for parts. Three of the required datasheets had false or misleading information. The first was the datasheet for the Nucleo dev-board housing the STM32. Repeatedly the team has had the datasheet claim one of the external morpho connecter pins were connected to a certain internal port or peripheral, only for that to turn out to be false. This led to a distrust in the datasheet leading to the team's greatest mistake: assigning both the USART TX and I2C SDA to the same pin. Eventually the team found the answer to the poor nucleo documentation: the schematic. The schematic allowed the team to cross reference the exact wiring inside the dev board allowing quicker progress. The second misleading datasheet was for the MP3 player. The MP3 player had many of its functionalities perform completely different from what the datasheet claimed. The most telling is for the commands that can be sent. For example, the command that was claimed to repeat a folder would only intermittently repeat a single file. On top of this, many functions were obfuscated and only available through third party arduino libraries. The only way around this was trial and error and sidestepping non-functional commands. The final case of poor documentation was the I2C OLED screen. This screen only has the datasheet stores with incorrect header and unavailable in all sites that allow its purchase. The team had to find it through an internet search, and are still unsure if it is the correct datasheet. This datasheet provided a non-working initialization sequence and failed to provide the correct driver chip for the OLED screen. The only way to solve this problem was trial and error.

This project constituted the first time we had built an I2C library; this came with many difficulties. Our first idea was to implement the library like a UART library with an RX and TX circular buffer. This caused many issues with providing arbitration on which buffer was allowed to use the peripheral. Any failed arbitration attempt would permanently lock up the I2C module. While there may have been a way to prevent all arbitration failures, it proved easier to restructure the library to have one I2C buffer that holds both receive requests and transmissions. This meant the request packets had to hold either a receive request or a transmit request indiscriminately. This created issues where the requirement to hold an unused pointer to the post function on every transmit request used the entire system memory. The large transmit buffer size was required by the OLED screen. The solution to this problem was to implement one pre-defined list of function pointers that can

be configured at compile time, then have each packet only hold the index the function is at. the greatly reduced the number of bytes each packet needed from 12 bytes to 4 bytes. This allows the buffer to be able to hold all RTC receive requests and nearly one and a half screens of OLED data.

The final lesson learned was to test early and test often. The embedded system had difficulties testing the components until the PCB was done. This could have been remedied by testing the code early with a bread board and dummy parts. This would have allowed more confidence in libraries and knowledge of issues and bugs earlier in the process. For example, the I2C module had arbitration issues with the RX and TX buffers that were unknown until the PCB was completed. Having the issue be found so late forced hacky solutions. This takes the shape of the defined pointer list, a far more elegant solution would be to have a hash table to store pointers given by code. however this would require early knowledge to develop and test the hash table library. Many other systems followed this same path of developing a library without testing it and finding issues far too late into development to implement the cleaner solution.

6.2 Future Improvements

Future iterations can improve housing ruggedness, add solar charging, fix the double use of pin PA1, and allow real-time logging via a mobile app.

One issue that must be addressed in future iterations is the double mapping of the STM's pin PA1. This issue was found too late into development to solely get a new version of the PCB to fix it. This bug necessitates that the GUI be robust to garbage data and the MCU to slow down to swap between the functionalities. Because the issue was found late, the solutions are rushed and prone to bugs that break both the USART and Buttons Menu. We are lucky that we implemented so many backups to operating the system as in case of USART and Buttons menu failure, the Bluetooth can still be used

7. Conclusion

7.1 Summary of Accomplishments

We successfully developed and tested a deployable wildlife playback speaker system that meets all core project requirements.

8. References

[1] Texas Instruments. LM2596 SIMPLE SWITCHER® Power Converter Datasheet. https://www.ti.com/product/LM2596

[2] STMicroelectronics. STM32WB0 Datasheet and Reference Manual. https://www.st.com/en/microcontrollers-microprocessors/stm32wb-series.html

[3] University of California, Santa Cruz. ECE 129 Capstone Guidelines. UCSC Engineering Department, 2024.